Сценарий защиты ПР1–ПР3

Проект: «Сервис управления подписками на уведомления» (репозиторий `fr1`)

Общее время: 10–15 минут. Структура: представление → демонстрация программы → тесты и качество кода → git-история → контрольные вопросы.

---

1. Представление проекта (30–60 сек)

> «Мой индивидуальный сквозной проект — консольный сервис управления
> подписками на уведомления. Пользователь (подписчик) может оформить
> подписку на тип уведомлений (акции, новости, системные оповещения)
> по одному из каналов — email, push или SMS. Приложение проверяет
> статус подписки, считает, сколько дней осталось до её окончания, и
> формирует текст уведомления. Проект развивался в три этапа: ПР1 —
> одиночный сценарий на переменных, ПР2 — функциональная модель на
> словарях и списках, ПР3 — объектная модель на классах.»

---

2. Демонстрация программы

Команды выполнять из папки `fr1`:

```
python main.py
```

Пройти вживую 3–4 пункта меню:

1. Пункт 1 — «Показать подписчиков»: показывает, что данные загрузились из `data/subscribers.json`.
2. Пункт 8 — «Оформить подписку»: ввести id существующего подписчика, id типа уведомлений, канал (например `email`), дату окончания в будущем. Показать сообщение об успехе.
3. Повторить пункт 8 с теми же параметрами — показать, что повторная активная подписка на тот же тип и канал запрещена (`create_subscription()` вернёт `None`).
4. Пункт 11 — «Показать уведомление по подписке»: продемонстрировать вывод статуса, количества дней и текста уведомления — `Subscription.get_status()`, `days_until_renewal()`, `format_notification()`.

5. Пункт 9 — «Отменить подписку», затем снова пункт 8 с теми же параметрами — показать, что после отмены оформить подписку снова можно (`is_active` стал `False`, канал освободился).

Если спросят «а как это было на ПР1/ПР2» — открыть в IDE историю файла (`git log --follow main.py` или посмотреть старые коммиты), либо просто показать код функции `check_subscription_status()` в `models/subscriptions.py` и сказать: «эта функция и три других (`validate_email`, `days_until_renewal`, `format_notification`) перенесены из ПР1 без изменений — только теперь их вызывают методы класса `Subscription`».

---

3. Тесты и качество кода

```
python -m pip install -r requirements.txt # если ещё не установлено
pytest -v
flake8
```

> «24 теста, все проходят. flake8 не выдаёт замечаний — код
> оформлен по PEP 8, аннотации типов на всех основных функциях и
> методах.»

Если преподаватель просит показать конкретный тест — открыть `tests/test_subscriptions.py`, показать `test_duplicate_active_subscription_forbidden` и `test_cancelled_subscription_frees_channel` — они прямо проверяют ключевое правило предметной области.

---

4. Git-история

```
git log --oneline
```

> «Видно последовательное развитие: первый коммит — стартовый
> сценарий ПР1, второй — отладка через breakpoint (шаг 6.6
> методички), третий — доработка до функциональной модели ПР2,
> два последних — переход на объектную модель в ПР3: сначала

> добавлены классы, потом на них переведены main.py, storage.py и

> тесты.»

Репозиторий на GitHub: https://github.com/sonic0666/fr1

---

5. Подготовленные ответы на вероятные контрольные вопросы

Классы и объекты

• *Чем класс отличается от объекта?* — Класс `Subscriber` описывает структуру (какие атрибуты и методы есть у любого подписчика), объект — конкретный подписчик с конкретными `name`/`email`, например загруженный из `data/subscribers.json`.

• *Что такое `self`?* — Ссылка на текущий объект внутри метода; через неё методы `Subscription` обращаются к своим же `end_date`, `is_active` и т. д.

• *Зачем `__str__`?* — Чтобы `print(subscription)` сразу выводил человекочитаемую строку со статусом, а не адрес объекта в памяти.

Атрибуты и инкапсуляция

• *Как в проекте проявляется инкапсуляция?* — Логика проверки канала и статуса подписки не разбросана по `main.py`, а находится внутри класса `Subscription` (`get_status()`, `days_until_renewal()`): внешний код не обязан знать детали расчёта, только вызывает метод.

staticmethod / classmethod

• *Зачем `@staticmethod` на `Subscriber.validate_email()`?* — Функция не использует данные конкретного подписчика (`self` не нужен), но логически относится к классу `Subscriber`, поэтому вызывается как `Subscriber.validate_email(email)`, а не через объект.

• *Зачем `@classmethod` на `Subscriber.from_data()`?* — Нужен доступ к самому классу (`cls`), чтобы создать новый объект `Subscriber` из словаря при загрузке JSON — это альтернативный способ создания объекта, отдельный от обычного `__init__`.

Связь объектов

• *Почему `Subscription` хранит объект `Subscriber`, а не просто его id?* — Чтобы можно было напрямую обращаться к данным подписчика: `subscription.subscriber.name`, без повторного поиска по id в коллекции.

Хранение данных

• *Почему данные лежат в JSON, а не в переменных?* — Чтобы они не терялись между запусками программы; `storage.py` отвечает за преобразование JSON → объекты при загрузке и объекты → JSON при сохранении.

Развитие проекта

• *Какие функции ПР2 стали методами в ПР3?* — `check_subscription_status` осталась отдельной функцией (её вызывает метод `get_status()`), а `days_until_renewal` и `format_notification` стали методами `Subscription`, потому что оперируют данными конкретного объекта (`self.end_date`, `self.subscriber`, `self.notification_type`).

• *Какие функции остались обычными?* — `add_subscriber`, `find_subscriber`, `create_subscription` и им подобные — они работают с целой коллекцией, а не с одним объектом, поэтому не стали методами класса.

Чек-лист перед защитой

- [] `python main.py` запускается без ошибок, меню отрабатывает
- [] `pytest -v` — все тесты зелёные
- [] `flake8` — пусто (без замечаний)
- [] `git log --oneline` показывает всю историю (5 коммитов)
- [ ] Репозиторий запушен на GitHub и доступен по ссылке
- [ ] Пройдены уроки Яндекс.Практикума по темам ПР1/ПР2/ПР3
- [ ] Готовы отчёты по ПР1, ПР2 и ПР3